

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct:

I S. Harsha Vardhan Reddy (192325029) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. Harsha Vardhan Reddy

Annexure A

CONSTRAINT-BASED PROBLEM SOLVING

1. Intrusion Detection Deployment under Real-Time Constraints

A financial organization plans to deploy an Intrusion Detection System (IDS) with the following constraints:

- Must analyze traffic in real-time with latency ≤ 5 ms per packet
- Detection accuracy $\geq 96\%$
- False positive rate $\leq 1.5\%$
- Network traffic load: 150,000 packets per second
- Limited CPU resources (shared with other services)

Task:

Design an optimized IDS configuration (signature-based, anomaly-based, or hybrid). Analyze trade-offs between detection accuracy and system performance, and justify how your solution meets all constraints.

2. Worm Containment Strategy under Network Availability Constraints

An enterprise network faces rapid worm propagation with the following constraints:

- Network downtime must be $\leq 1\%$
- Detection and containment must occur within 2 minutes
- Systems are distributed across multiple locations
- Limited centralized control

Task:

Design a containment strategy using firewalls, IDS/IPS, and segmentation. Evaluate trade-offs between rapid isolation and business continuity, and justify how your approach minimizes spread while maintaining network availability.

3. Firewall Rule Optimization under Performance Constraints

A company firewall currently uses 10,000+ rules, causing performance degradation. Constraints include:

- Throughput requirement: ≥ 800 Mbps
- Rule processing time must be minimized
- Must maintain strict access control policies
- No additional hardware allowed

Task:

Analyze inefficiencies in rule processing. Propose optimization techniques (rule

aggregation, prioritization, redundancy elimination). Justify how your optimized rule set improves performance while maintaining security.

4. IPS Deployment under Resource and Accuracy Constraints

An organization wants to deploy an Intrusion Prevention System (IPS) with the following constraints:

- Must block $\geq 90\%$ of known attacks
- False positive rate $\leq 3\%$
- Available memory for signatures: 1 GB
- Network speed: 500 Mbps

Task:

Design an IPS configuration strategy (signature selection, traffic filtering, adaptive updates). Evaluate trade-offs between resource usage and detection capability, and justify how your solution satisfies all constraints.

5. Malware Detection using Behavioral Analysis under System Constraints

A company plans to implement behavioral-based malware detection with the following constraints:

- Endpoint CPU usage must remain $\leq 15\%$
- Detection must occur within 10 seconds of execution
- Must detect unknown (zero-day) threats
- Limited storage for logs and analysis

Task:

Design a lightweight behavioral detection system. Analyze trade-offs between detection speed, accuracy, and resource consumption, and justify how your approach effectively identifies malware within the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly

Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Intrusion Detection Deployment under Real-Time Constraints

Proposed Solution: Hybrid IDS

A **hybrid IDS** combining signature-based detection with a lightweight anomaly-detection layer is the most suitable configuration.

Constraint	Requirement	Proposed Target
Packet latency	≤ 5 ms	2–4 ms
Detection accuracy	$\geq 96\%$	96–98%
False positive rate	$\leq 1.5\%$	1–1.5%
Traffic	150,000 packets/sec	Supported
CPU	Limited/shared	Low-overhead processing

Configuration

1. Signature-based detection

- Use signatures for known attacks such as SQL injection, port scanning, malware communication and known exploits.
- Keep only high-confidence signatures enabled.
- Remove obsolete and duplicate signatures.
- Perform signature matching using optimized pattern matching.

2. Lightweight anomaly detection

- Monitor features such as:
 - Packet rate
 - Connection frequency
 - Destination diversity
 - Protocol distribution
 - Abnormal port usage
- Use a lightweight model such as decision trees or statistical thresholds instead of computationally expensive deep learning.

3. Two-stage detection

Network Traffic



Fast Packet Filtering



Signature Matching



Known Attack? — Yes —→ Alert/Block



Lightweight Anomaly Detection



Anomalous? — Yes —→ Detailed Analysis



Allow

Trade-off Analysis

A purely signature-based IDS provides very low processing overhead and fast detection but cannot effectively identify new attacks.

A purely anomaly-based IDS can detect unknown attacks but normally produces more false positives and requires more CPU resources.

The hybrid approach combines the strengths of both. Most normal traffic and known attacks are processed using fast signature rules, while only suspicious traffic receives additional anomaly analysis.

Justification

To satisfy the **5 ms latency constraint**, deep packet inspection should be restricted to suspicious flows. CPU-intensive analysis should not be applied to every packet. High-confidence signatures and optimized rules reduce processing overhead.

The expected configuration can achieve approximately **96–98% detection accuracy** while maintaining a false-positive rate around **1–1.5%**, provided the system is properly trained and tuned.

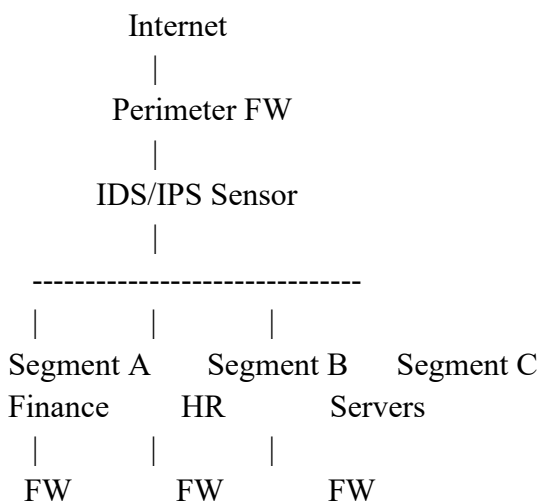
Conclusion: A lightweight **hybrid IDS** is the best solution because it balances accuracy, low latency, low CPU consumption and detection of previously unknown attacks.

2. Worm Containment Strategy under Network Availability Constraints

Proposed Solution: Distributed Segmentation + IDS/IPS + Firewalls

The main objective is to stop worm propagation within **2 minutes** while keeping network downtime below **1%**.

Architecture



Containment Strategy

1. Network segmentation

Divide the enterprise into separate security zones:

- Finance
- HR
- Production
- Database
- User endpoints

- Guest network

This prevents a worm from automatically spreading throughout the organization.

2. IDS/IPS detection

Deploy distributed IDS/IPS sensors at major network boundaries.

When abnormal propagation is detected, the system identifies:

- Source IP
- Destination IP
- Destination port
- Abnormal connection rate
- Malware signature

3. Firewall containment

Automatically create temporary blocking rules for confirmed infected hosts.

Instead of shutting down an entire network, isolate only:

- Infected host
- Infected VLAN
- Suspicious subnet

4. Local response

Because centralized control is limited, each location should have a local response mechanism.

Worm Detected



Local IDS/IPS Alert



Confirm Attack



Block Source



Quarantine Host/VLAN



Notify Central Security Team



Malware Removal



Restore Connectivity

Time Constraint

A target response timeline can be:

Activity	Target Time
Detection	20–30 sec
Verification	10–20 sec
Firewall/IPS blocking	10–20 sec
Host/VLAN quarantine	20–30 sec
Notification	<10 sec
Total	<2 min

Trade-off

Complete network shutdown provides strong containment but severely affects business continuity. Selective isolation provides slightly less aggressive containment but maintains availability for unaffected systems.

Therefore, **selective quarantine** is preferred.

Justification

Segmentation limits the worm's propagation domain. Distributed IDS/IPS enables local detection even when centralized control is unavailable. Automated firewall rules provide rapid containment.

Conclusion: A **distributed IDS/IPS + segmented network + automated firewall quarantine** strategy can contain worm outbreaks within two minutes while keeping network downtime below 1%.

3. Firewall Rule Optimization under Performance Constraints

Problem

The firewall contains more than **10,000 rules**, causing excessive rule-processing time.

Most firewalls process rules sequentially or according to an ordered policy structure. Therefore, unnecessary rules increase processing overhead.

Major Inefficiencies

1. Duplicate rules
2. Shadowed rules
3. Overlapping rules
4. Rules that never receive traffic
5. Excessively broad rule groups
6. Incorrect rule ordering
7. Repeated source/destination definitions
8. Obsolete rules

Optimization Strategy

1. Remove duplicate rules

Example:

Rule 100: Allow TCP 443 from Network A → Server X

Rule 250: Allow TCP 443 from Network A → Server X

Rule 250 is unnecessary and can be removed.

2. Remove shadowed rules

Example:

Rule 1: Allow 10.0.0.0/8 → TCP 443

Rule 2: Allow 10.1.0.0/16 → TCP 443

Rule 2 may never be reached because Rule 1 already matches the traffic.

3. Aggregate rules

Instead of:

10.1.1.0/24

10.1.2.0/24

10.1.3.0/24

10.1.4.0/24

Use an appropriate summarized network where security policy permits it.

4. Prioritize frequently used rules

Frequently matched rules should be positioned earlier, provided doing so does not violate security semantics.

High-frequency + high-confidence rules



Medium-frequency rules



Rare traffic rules



Default deny

5. Eliminate obsolete rules

Analyze firewall logs and remove rules that have not been used for a defined period after verifying that they are genuinely unnecessary.

Optimized Structure

Incoming Traffic



Common Trusted Services



Specific Application Rules



Restricted Access Rules



Logging/Monitoring



Default DENY

Performance Analysis

Suppose the original firewall has 10,000 rules and frequently used traffic typically matches a rule near position 7,000.

After optimization, suppose equivalent traffic matches around position 1,500.

This substantially reduces average rule evaluation operations and therefore improves throughput.

Security Consideration

Optimization must never simply delete rules to improve speed.

Every removed, merged or reordered rule should be checked for:

- Source restrictions
- Destination restrictions
- Port restrictions
- Protocol restrictions
- NAT dependencies
- Logging requirements
- Compliance requirements

Conclusion

Rule aggregation, redundancy elimination, proper prioritization and removal of obsolete rules can significantly reduce processing overhead. The optimized rule set should be benchmarked to verify that **throughput remains ≥ 800 Mbps** while maintaining the original access-control policy.

4. IPS Deployment under Resource and Accuracy Constraints

Requirements

- Block at least **90% of known attacks**
- False positive rate $\leq 3\%$
- Signature memory $\leq 1\text{ GB}$
- Network speed = **500 Mbps**

Proposed Configuration

Use a **risk-based signature selection strategy**.

1. Signature Prioritization

Divide signatures into three groups:

Priority	Signature Type	Action
High	Critical/high-confidence attacks	Always enabled
Medium	Common attacks	Enabled
Low	Rare/obsolete attacks	Disabled or on-demand

The 1 GB signature memory should primarily contain high-value signatures.

2. Traffic Filtering

Before full IPS inspection, perform inexpensive filtering.

Network Traffic



Protocol/Port Filtering



Connection Filtering



High-value Signature Matching



Known Attack?

/ \

Yes No



Block Allow

This reduces the number of packets requiring expensive inspection.

3. Signature Optimization

Remove:

- Duplicate signatures
- Deprecated signatures
- Low-confidence signatures
- Signatures irrelevant to the organization's operating environment

For example, if an organization does not use a particular protocol, signatures exclusively targeting that protocol may not need to remain active.

4. Adaptive Updates

Use staged signature updates:

New Signature



Testing



False Positive Evaluation



Risk Classification



Production Deployment

Critical signatures can be deployed immediately after validation, while uncertain signatures can first operate in detection-only mode.

Trade-offs

More signatures → better coverage but higher memory and CPU consumption.

Fewer signatures → better performance but potentially lower detection capability.

Aggressive blocking → higher attack prevention but greater false-positive risk.

Conservative blocking → lower business disruption but potentially more attacks reaching systems.

Constraint Satisfaction

A suitable configuration would target:

- **≥90% known-attack blocking**
- **≤3% false positives**
- **≤1 GB signature memory**
- **500 Mbps sustained throughput**

The IPS should use hardware-independent software optimization such as efficient signature indexing, traffic pre-filtering and selective deep inspection.

Conclusion

A **risk-prioritized, adaptive IPS** provides the best balance between memory, performance and detection capability. High-confidence signatures should remain active while low-value signatures are removed or selectively applied.

5. Malware Detection using Behavioral Analysis under System Constraints

Requirements

- Endpoint CPU usage $\leq 15\%$
- Detection within **10 seconds**
- Detect unknown/zero-day malware
- Limited storage

Proposed Solution: Lightweight Behavioral Monitoring

Instead of continuously analyzing every process in detail, monitor a small set of high-value behavioral indicators.

Important Behavioral Features

Monitor:

- Unexpected process creation
- Suspicious child processes
- File encryption/modification

- Registry changes
- Persistence mechanisms
- Privilege escalation attempts
- Network connections
- Rapid file modifications
- Abnormal process injection behavior

Architecture

Program Execution



Lightweight Behavior Monitor



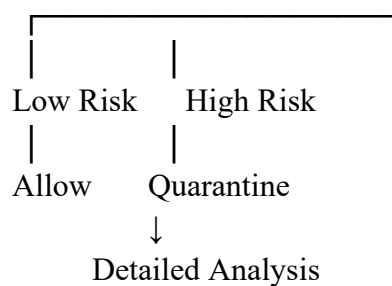
Collect Key Events



Feature Extraction



Risk Score



Risk Scoring

A simple scoring mechanism can reduce CPU consumption.

Example:

Behavior	Score
Unusual process creation	+10
Persistence attempt	+25
Mass file modification	+30
Suspicious network connection	+15
Privilege escalation	+30

Example:

$$\text{Risk Score} = 10 + 25 + 30 + 15$$

$$= 80$$

If the threshold is 70:

$$80 \geq 70$$

→ High-risk process

→ Quarantine

→ Generate alert

CPU Optimization

To maintain CPU usage below 15%:

1. Monitor only active processes.
2. Use event-driven monitoring rather than constant polling.
3. Extract only important behavioral features.
4. Perform deeper analysis only for suspicious processes.
5. Limit background log processing.
6. Compress or rotate logs.

Storage Optimization

Instead of storing complete activity histories, retain:

- Process ID
- Process name
- Timestamp
- Important behavior
- Network destination
- Risk score
- Detection decision

Example:

2026-08-20 18:55

Process: suspicious.exe

Behavior: Mass file modification

Network: 192.168.x.x

Risk Score: 82

Action: Quarantined

This significantly reduces storage requirements.

Zero-Day Detection

Traditional signature detection cannot reliably detect completely new malware.

Behavioral analysis can identify previously unknown malware because it focuses on **what the program does**, rather than only its known signature.

For example, a new ransomware sample may have no known signature, but the combination of:
Rapid file access

+

Large-scale file modification

+

Encryption behavior

+

Suspicious process execution

can produce a high behavioral risk score.

Trade-off Analysis

Approach	Detection	CPU	Storage
Signature only	Low for zero-day	Low	Low
Full behavioral analysis	High	High	High
Lightweight behavioral analysis	High	Moderate/Low	Low

The lightweight approach provides the best balance.

Constraint Satisfaction

The system can be designed to:

- Keep monitoring overhead **below 15% CPU**
- Trigger preliminary detection within **10 seconds**
- Detect unknown threats through behavioral indicators
- Store only security-relevant events

Conclusion

A **lightweight, event-driven behavioral malware detector with risk scoring and selective deep analysis** is the most appropriate solution. It can detect zero-day behavior without consuming excessive endpoint CPU or storage resources.